Chapter 4 – MongoDB Service

JoinDevOps Roboshop – CentOS Stream 9 Edition

4.1 Objective

MongoDB is the NoSQL database used by the **Catalogue** and **User** services in the RoboShop application. In this chapter, you will install MongoDB on CentOS Stream 9, configure it for remote access, and prepare it for the application.

4.2 Why MongoDB?

MongoDB stores data in flexible JSON-like documents instead of tables.

In RoboShop:

- **Catalogue Service** stores product information.
- **User Service** stores user account information.

These services communicate directly with MongoDB whenever they need to read or update data.

4.3 Prerequisites

Before proceeding, verify that:

- Chapter 2 (Environment Preparation) is completed.
- Frontend deployment is complete.
- Internet connectivity is available.

Verify the operating system:

```
cat /etc/redhat-release
```

Expected output:

```
CentOS Stream release 9
```

4.4 Configure the MongoDB Repository

Create the MongoDB repository file.

```
vi /etc/yum.repos.d/mongodb-org-7.0.repo
```

Add the following configuration:

```
[mongodb-org-7.0]
name=MongoDB Repository
baseurl=https://repo.mongodb.org/yum/redhat/9/mongodb-org/7.0/x86_64/
enabled=1
gpgcheck=1
gpgkey=https://pgp.mongodb.com/server-7.0.asc
```

Save the file.

Verify:

```
cat /etc/yum.repos.d/mongodb-org-7.0.repo
```

4.5 Install MongoDB

Install both the MongoDB server and the MongoDB Shell.

```
dnf install -y mongodb-org mongodb-mongosh
```

Verify the installation:

```
mongod --version
```

```
mongosh --version
```

4.6 Start the MongoDB Service

Enable the service to start automatically after reboot.

```
systemctl enable mongod
```

Start the service.

```
systemctl start mongod
```

Verify:

```
systemctl status mongod
```

Expected output:

```
active (running)
```

4.7 Configure Remote Connectivity

Open the MongoDB configuration file.

```
vi /etc/mongod.conf
```

Locate:

```
bindIp: 127.0.0.1
```

Replace it with:

```
bindIp: 0.0.0.0
```

Save the file.

Restart MongoDB.

```
systemctl restart mongod
```

4.8 Verify MongoDB

Check whether MongoDB is listening on port **27017**.

```
ss -lntp | grep 27017
```

Connect to MongoDB.

```
mongosh
```

Exit the shell.

```
exit
```

4.9 Schema Preparation

Later, when the **Catalogue** and **User** services are installed, they will load their database schemas into MongoDB.

Typical commands are:

```
mongosh < /app/db/master-data.js
```

or

```
mongosh < /app/schema/catalogue.js
```

depending on the service being configured.

4.10 Verification Checklist

Confirm the following:

- MongoDB repository created.
- MongoDB packages installed.
- MongoDB service enabled.
- MongoDB service running.
- Port **27017** listening.
- Remote connections allowed.
- `mongosh` installed successfully.

4.11 Troubleshooting

MongoDB service fails to start

Check:

```
journalctl -u mongod -xe
```

Port 27017 is not listening

Verify:

```
systemctl status mongod
```

Then check:

```
ss -lntp | grep 27017
```

Unable to connect remotely

Verify:

- `bindIp` is set to `0.0.0.0`
- Firewall rules allow TCP 27017.
- The MongoDB service is running.

`mongosh` **command not found**

Install it using:

```
dnf install -y mongodb-mongosh
```

4.12 CentOS Stream 9 Notes

Compared to older Roboshop deployments:

- Use **MongoDB 7.x**.
- Use `mongosh` instead of the legacy `mongo` shell.
- Install `mongodb-mongosh` along with the server.
- Use the **EL9 MongoDB repository**.

4.13 Interview Corner

1. Why does RoboShop use MongoDB?
2. Which services connect to MongoDB?
3. Why is `bindIp` changed to `0.0.0.0`?
4. What is the difference between `mongo` and `mongosh`?
5. How do you verify that MongoDB is accepting connections?
6. How would you troubleshoot a MongoDB service that fails to start?

Chapter Summary

In this chapter, you:

- Configured the MongoDB EL9 repository.
- Installed MongoDB Server and MongoDB Shell.
- Enabled and started the service.
- Allowed remote connectivity.
- Verified the installation.
- Prepared MongoDB for the Catalogue and User services.

In the next chapter, we will deploy the **Catalogue Service**, which uses Node.js and stores product data in MongoDB.